

Presentación Final del Proyecto

Informe Final — DevSecOps

FinSight

Gestor de finanzas personales *privacy-first* para LATAM con auto-categorización por IA

<https://github.com/bylenz/finsight>

Curso: DevOps — UTEC • **Semana 16** • **Valor:** 30% de la nota final

Rol	Integrante
Tech Lead / DevOps	Lenin Chávez
Backend & Security Lead	Jerimy Sandoval
AI & Analytics Lead	Adrian Céspedes
Frontend Dev	Lenin Chávez, Adrian Céspedes
QA / Test Strategy	Jerimy Sandoval

1. Objetivo

Exponer de forma sintética y clara los resultados obtenidos a lo largo del proyecto integrador FinSight, destacando la aplicación práctica de los conceptos de **DevOps** y **DevSecOps**: modelado de amenazas STRIDE, integración de análisis de seguridad (SAST, DAST, secret scanning y SCA) en el pipeline de CI/CD, centralización de hallazgos en DefectDojo, y las lecciones aprendidas y recomendaciones para futuras implementaciones en entornos de desarrollo ágil y continuo.

2. Título y descripción del proyecto

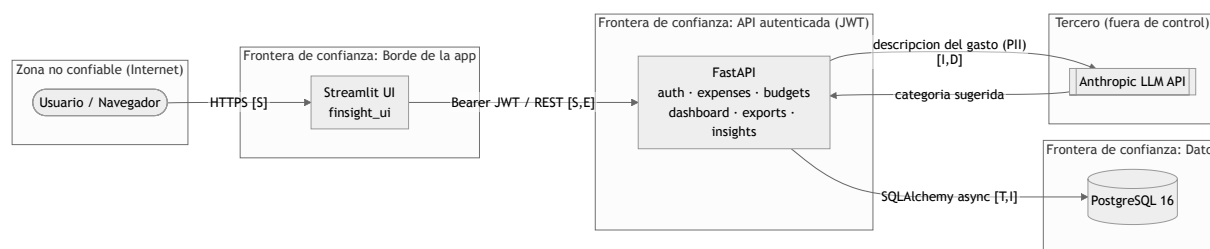
FinSight es un *personal finance tracker* basado en web con IA conversacional en español, construido para el mercado latinoamericano. Los usuarios registran gastos por voz, texto o CSV; la IA los categoriza automáticamente, detecta patrones de gasto y emite alertas de presupuesto. La aplicación es *privacy-first* (no requiere credenciales bancarias) y soporta multi-moneda (PEN, USD) y hogares multi-usuario.

Arquitectura (resumen):

- **Frontend:** Streamlit (`frontend/src/finsight_ui`).
- **Backend:** FastAPI (`backend/src/finsight`) — módulos `auth`, `expenses`, `budgets`, `categories`, `households`, `dashboard`, `exports`, `insights`.
- **Base de datos:** PostgreSQL 16 (SQLAlchemy async + Alembic).
- **Autenticación:** JWT (python-jose) + bcrypt (12 rounds).
- **Servicio externo:** Anthropic LLM para categorización de gastos.
- **Infraestructura:** Docker Compose; CI/CD en GitHub Actions.

3. Diagrama de modelado de amenazas (STRIDE)

Diagrama de flujo de datos con fronteras de confianza (*trust boundaries*). Cada arista se anota con las categorías STRIDE aplicables. Versión editable en draw.io: `docs/informe-final/stride-finsight.draw`



S=Spoofing • T=Tampering • R=Repudiation • I=Information Disclosure • D=Denial of Service • E=Elevation of Privilege

4. Cinco vulnerabilidades principales (STRIDE)

Threat Type	Componente	Descripción de la amenaza
Spoofing	Login / JWT	Reuso de un JWT robado o falsificado para suplantar a un usuario. El token incluía <code>jwt</code> pero no existía lista de revocación y vivía 24h; la seguridad dependía además del <code>SECRET_KEY</code> .
Tampering	expenses / budgets	Broken Object-Level Authorization (IDOR): un miembro podía leer o modificar recursos de otro hogar alterando el <code>id</code> en la petición si faltaban chequeos de pertenencia.
Info. Disclosure	insights (LLM) / exports	Las descripciones de gastos (PII) se envían a un tercero (Anthropic) para categorizar; ese dato sale del perímetro. Una exportación CSV mal autorizada podría filtrar gastos de otro hogar.
Denial of Service	FastAPI / LLM	Sin rate limiting: un atacante satura la API o abusa de la categorización por IA (cada gasto nuevo dispara una llamada al LLM), generando <i>cost-based DoS</i> .
Elev. of Privilege	households / dependencias	Un rol <code>viewer</code> ejecutando acciones de <code>owner</code> ; o una dependencia vulnerable (p. ej. CVE en <code>python-jose</code>) con ruta a ejecución de código y escalamiento.

Repudiation (brecha identificada): no existía *audit logging* de eventos de seguridad, por lo que las acciones no eran trazables ni atribuibles.

5. Elección de herramientas SAST y DAST

SAST (análisis estático): `Semgrep` (reglas `p/ci`, `p/security-audit`, `p/python`) + `Bandit` (analizador específico de Python).

DAST (análisis dinámico): `OWASP ZAP baseline scan` contra el contenedor del backend levantado en CI.

Evidencia de integración: las tres herramientas corren como *jobs* del workflow `.github/workflows/security` (GitHub Actions) — ver §11. Cada una exporta un reporte JSON (`semgrep-report.json`, `bandit-report.json`, `zap-report.json`) que se publica como artefacto de CI y se centraliza en DefectDojo (§8). Ejecución local reproducible: `Semgrep` y `Bandit` generaron hallazgos de severidad baja sobre `backend/src/finsight`,

visibles en el dashboard de la 8.

6. Elección de herramientas de Secret Scanning y SCA

Secret Scanning: Gitleaks + TruffleHog (`-results=verified,unknown`), más el hook `detect-private-key` de *pre-commit* como primera barrera local.

SCA (dependencias): Trivy (`filesystem` sobre las dependencias + escaneo de la imagen Docker) y generación de **SBOM** en formato CycloneDX.

Doble barrera: *pre-commit* bloquea secretos localmente y el pipeline vuelve a validarlos en CI.

Evidencia: en la ejecución real Gitleaks reportó *sin fugas*; TruffleHog, Trivy y el SBOM produjeron sus reportes (`gitleaks-report.json`, `trufflehog-report.json`, `trivy-fs-report.json`, `sbom.json`) e ingresaron a DefectDojo.

7. Centralización de vulnerabilidades en DefectDojo

Se desplegó **DefectDojo** localmente con Docker Compose (puerto `:8082`) y se centralizaron todos los hallazgos de los escáneres en un único dashboard de deuda técnica de seguridad. Jerarquía: Product Type **Aplicaciones Web** → Product **FinSight** → Engagement **CI Security Scan** → un Test por herramienta. La ingesta usa el endpoint `POST /api/v2/reimport-scan/` (deduplicación con `close_old_findings`) mediante el script `scripts/upload_to_defectdojo.sh`.

The screenshot displays the DefectDojo FinSight dashboard. At the top, there's a navigation bar with the DefectDojo logo, a search bar, and user profile icons. Below the navigation bar, a banner mentions a 25% discount on DefectDojo Pro and a redesigned UI beta opt-in. The main content area is titled 'FinSight' and includes a sidebar with navigation options: Overview, Components, Metrics, Engagements (1), Findings (22), Hosts / Endpoints (0/0), Benchmarks, and Settings.

The 'Findings' section is expanded, showing a description: 'Privacy-first LATAM personal-finance tracker'. Below this, the 'Metrics' section displays a bar chart with the following data:

Severity	Count
CRITICAL	16
HIGH	0
MEDIUM	0
LOW	6
INFORMATIONAL	0
TOTAL	22

Below the metrics, there are sections for 'Technologies' (no technologies found), 'Regulations' (no regulations found), 'Benchmark Progress' (no benchmarks found), and 'Authorized Users' (no authorized users found).

The right sidebar contains several sections:

- Metadata:** A table with fields like Business Criticality, Organization, Platform, Lifecycle, Origin, User Records, and Revenue, all marked as 'Not Specified'.
- Service Level Agreement:** A table showing remediation timeframes for different severity levels: Critical (7 days), High (30 days), Medium (90 days), and Low (120 days).
- Contacts:** A table with fields for Team Manager, Asset Manager, and Technical Contact, all marked as 'Unknown'.
- Notifications:** A table with a list of events (Engagement added, Close engagement, Test added, Scan added, SLA breach, SLA breach (combined), Risk Acceptance Expiration) and a checkbox for each, all currently unchecked.

At the bottom of the page, there's a footer with the DefectDojo logo, version information (v. 3.0.200), and copyright details (© 2015-2020 DefectDojo, Inc.).

Figura 1: DefectDojo — Product FinSight: 22 hallazgos por severidad y 5 escaneos (Gitleaks, TruffleHog, Trivy, Semgrep, Bandit).

DEFECTDOJO

Save 25% off DefectDojo Pro

A redesigned UI is available as a beta opt-in. It will become the default on September 8th in the 3.3.0 release. [Enable it in your profile.](#)

Home / Findings

Open Findings

Showing entries 1 to 22 of 22

Column visibility Copy PDF Print Search:

	Severity	Name	CWE	Vulnerability Id	EPSS Score	EPSS Percentile	Known Exploited	Used in Ransomware	Date Added to KEV	Date	Age	SLA	Reporter	Found By	Status	Group	Asset	Ser
☐	Critical	Hard Coded URI Secret In: /repo/.venv/lib/python3.12/site-p...	798		N.A.	N.A.	No	No		July 3, 2026	0	7	Admin User (admin)	Trufflehog Scan	Active		FinSight	
☐	Critical	Hard Coded Postgres Secret In: /repo/reports/trufflehog-rep...	798		N.A.	N.A.	No	No		July 3, 2026	0	7	Admin User (admin)	Trufflehog Scan	Active		FinSight	
☐	Critical	Hard Coded Postgres Secret In: /repo/reports/trufflehog-rep...	798		N.A.	N.A.	No	No		July 3, 2026	0	7	Admin User (admin)	Trufflehog Scan	Active		FinSight	
☐	Critical	Hard Coded Postgres Secret In: /repo/reports/trufflehog-rep...	798		N.A.	N.A.	No	No		July 3, 2026	0	7	Admin User (admin)	Trufflehog Scan	Active		FinSight	
☐	Critical	Hard Coded Postgres Secret In: /repo/reports/trufflehog-rep...	798		N.A.	N.A.	No	No		July 3, 2026	0	7	Admin User (admin)	Trufflehog Scan	Active		FinSight	
☐	Critical	Hard Coded URI Secret In: /repo/.venv/lib/python3.12/site-p...	798		N.A.	N.A.	No	No		July 3, 2026	0	7	Admin User (admin)	Trufflehog Scan	Active		FinSight	
☐	Critical	Hard Coded URI Secret In: /repo/.venv/lib/python3.12/site-p...	798		N.A.	N.A.	No	No		July 3, 2026	0	7	Admin User (admin)	Trufflehog Scan	Active		FinSight	
☐	Critical	Hard Coded Postgres Secret In: /repo/.venv/lib/python3.12/s...	798		N.A.	N.A.	No	No		July 3, 2026	0	7	Admin User (admin)	Trufflehog Scan	Active		FinSight	
☐	Critical	Hard Coded Postgres Secret In: /repo/.venv/lib/python3.12/s...	798		N.A.	N.A.	No	No		July 3, 2026	0	7	Admin User (admin)	Trufflehog Scan	Active		FinSight	
☐	Critical	Hard Coded URI Secret In: /repo/.venv/lib/python3.12/site-p...	798		N.A.	N.A.	No	No		July 3, 2026	0	7	Admin User (admin)	Trufflehog Scan	Active		FinSight	
☐	Critical	Hard Coded URI Secret In: /repo/.venv/lib/python3.12/site-p...	798		N.A.	N.A.	No	No		July 3, 2026	0	7	Admin User (admin)	Trufflehog Scan	Active		FinSight	
☐	Critical	Hard Coded Postgres Secret In: /repo/.venv/lib/python3.12/s...	798		N.A.	N.A.	No	No		July 3, 2026	0	7	Admin User (admin)	Trufflehog Scan	Active		FinSight	
☐	Critical	Hard Coded Postgres Secret In: /repo/.venv/lib/python3.12/s...	798		N.A.	N.A.	No	No		July 3, 2026	0	7	Admin User (admin)	Trufflehog Scan	Active		FinSight	
☐	Critical	Hard Coded Postgres Secret In: /repo/.venv/lib/python3.12/s...	798		N.A.	N.A.	No	No		July 3, 2026	0	7	Admin User (admin)	Trufflehog Scan	Active		FinSight	
☐	Critical	Hard Coded URI Secret In: /repo/reports/trufflehog-report;...	798		N.A.	N.A.	No	No		July 3, 2026	0	7	Admin User (admin)	Trufflehog Scan	Active		FinSight	
☐	Critical	Hard Coded Postgres Secret In: /repo/reports/trufflehog-rep...	798		N.A.	N.A.	No	No		July 3, 2026	0	7	Admin User (admin)	Trufflehog Scan	Active		FinSight	
☐	Low	Possible Hardcoded Password: 'Bearer'			N.A.	N.A.	No	No		July 3, 2026	0	120	Admin User (admin)	Bandit Scan	Active		FinSight	
☐	Low	Possible Hardcoded Password: 'Access'			N.A.	N.A.	No	No		July 3, 2026	0	120	Admin User (admin)	Bandit Scan	Active		FinSight	
☐	Low	Possible Hardcoded Password: 'Refresh'			N.A.	N.A.	No	No		July 3, 2026	0	120	Admin User (admin)	Bandit Scan	Active		FinSight	
☐	Low	Try, Except, Pass Detected.			N.A.	N.A.	No	No		July 3, 2026	0	120	Admin User (admin)	Bandit Scan	Active		FinSight	
☐	Low	Use of Assert Detected. The Enclosed Code Will Be Removed W...			N.A.	N.A.	No	No		July 3, 2026	0	120	Admin User (admin)	Bandit Scan	Active		FinSight	
☐	Low	Possible Hardcoded Password: 'Bearer'			N.A.	N.A.	No	No		July 3, 2026	0	120	Admin User (admin)	Bandit Scan	Active		FinSight	

Showing entries 1 to 22 of 22



v. 3.0.200 (release mode)

Detailed DefectDojo documentation can be found in our GitHub Pages.

DefectDojo is licensed under the 3-Clause BSD License. | Dependencies Notices.

© 2015-2025 DefectDojo, Inc. All rights reserved. DefectDojo is trademark of DefectDojo, Inc.

Figura 2: DefectDojo — listado de hallazgos abiertos centralizados.

Nota: los 16 hallazgos *Critical* son candidatos de TruffleHog (`-results=verified,unknown`), en su mayoría sin verificar (tokens de ejemplo / fixtures de test), no secretos reales en producción.

8. Tabla de mitigaciones (hallazgos priorizados)

ID	Descripción del hallazgo	Sev.	Herramienta	Recomendación / estado
CRIT-001	Posible CVE en dependencia transitiva con ruta a ejecución de código	CRITICAL	Trivy / pip-audit	Actualizar a versión segura; fijar en el lock. <i>Detectado por pipeline.</i>
HIGH-001	IDOR en <code>expenses/budgets</code> entre hogares	HIGH	Semgrep / revisión	Validar <code>household_id/user_id</code> . <i>Mitigado (PR #21).</i>
HIGH-002	JWT sin revocación; token filtrado válido 24h	HIGH	STRIDE	Denylist por <code>jti</code> + refresh + TTL 15 min. <i>Mitigado (PR #23).</i>
MED-001	PII enviada al LLM sin minimización	MEDIUM	STRIDE / DAST	Minimizar/anonimizar; opt-in. <i>Parcial (PR #22).</i>
MED-002	API sin rate limiting → DoS / abuso de costo LLM	MEDIUM	OWASP ZAP	SlowAPI + circuit breaker. <i>Mitigado (PR #22).</i>
LOW-001	Sin secret scanning en CI previo	LOW	Gitleaks / TruffleHog	Integrar en pre-commit y pipeline. <i>Mitigado (PR #19).</i>

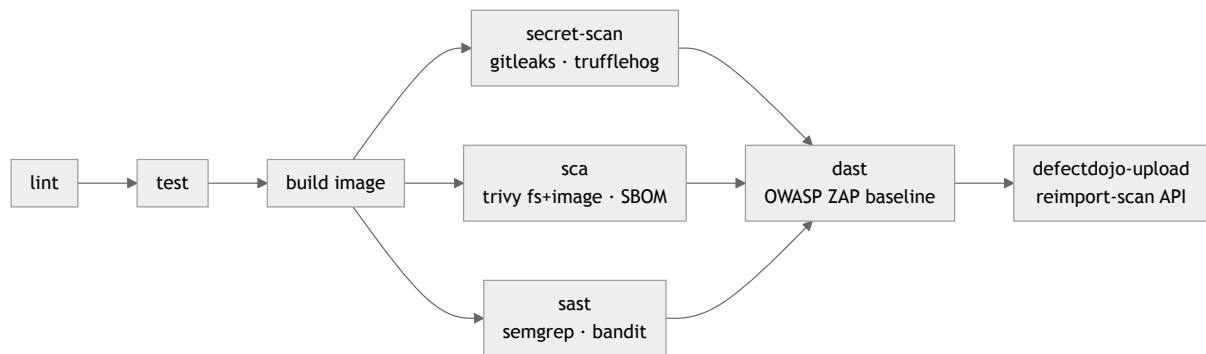
9. Situación inicial vs. resultados

Métrica	Antes	Después
Herramientas de seguridad en CI	0	5 (Gitleaks, TruffleHog, Trivy+SBOM, Semgrep, Bandit) + ZAP (DAST)
Hallazgos visibles / centralizados	0 (invisibles)	22 en DefectDojo (16 Critical, 6 Low)
Gestión de vulnerabilidades	Ninguna	Dashboard DefectDojo por severidad/herramienta
Barreras contra secretos	1 (detect-private-key)	3 (detect-private-key + Gitleaks + TruffleHog)
Vulnerabilidades STRIDE mitigadas en código	0	4 (IDOR, rate-limit, refresh tokens, audit log)

Situación inicial: el CI sólo ejecutaba `lint + test`; las vulnerabilidades eran *invisibles*. **Resultados:** pipeline con etapas de seguridad, hallazgos centralizados y priorizados en DefectDojo, doble barrera de secretos, y 4 amenazas STRIDE mitigadas en el código.

10. Pipeline CI/CD completo

10.1 Diagrama del pipeline



Los jobs `lint` y `test` permanecen en `ci.yml`. Los escaneos ligeros (`secret-scan`, `SAST`, `SCA` de filesystem) corren en cada push/PR; los pesados (`ZAP DAST`, `Trivy image`) se limitan a `main` y a un `schedule` semanal. Todos los jobs son no bloqueantes (`allow_failure-first`).

10.2 Evidencia: ejecución real en GitHub Actions

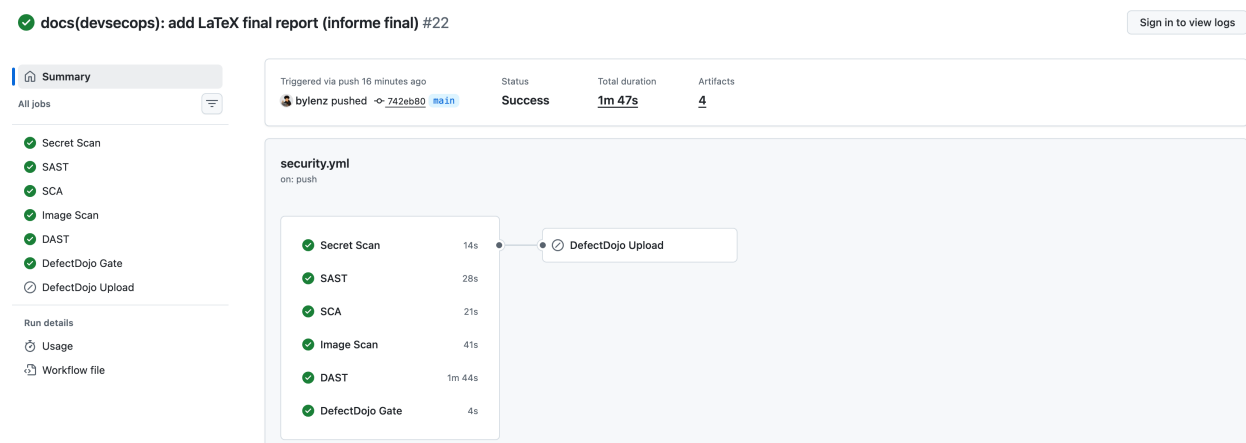


Figura 3: Run del workflow `security.yml` en GitHub Actions (push a `main`): estado *Success* en 1m47s, con el grafo de jobs Secret Scan, SAST, SCA, Image Scan, DAST y DefectDojo Gate → DefectDojo Upload (este último *skipped*: sin secreto configurado, comportamiento *evidence-on-demand*).

Artifacts		
Produced during runtime		
Name	Size	Digest
image-scan-reports	149 KB	sha256:68ed62863e9bfe79a8e4f7ab5d595cd1ad568c587a6eeb...
sast-reports	2.88 KB	sha256:2ee9f1708b3e63b5ba52f289ca9fe6483b55b7468f2b4c...
sca-reports	1.73 KB	sha256:fe59290fb268f5278c7a78927324f6092268885176eb1d...
secret-scan-reports	297 Bytes	sha256:3688f7e456a7b7a18de19f97b46996f4e78dd64d3da82...

Figura 4: Artefactos generados por el pipeline: reportes de los escáneres (`image-scan-reports`, `sast-reports`, `sca-reports`, `secret-scan-reports`).

10.3 Script de configuración (`.github/workflows/security.yml`)

Extracto representativo (archivo completo en el repositorio):

```

name: security
on:
  push: { branches: [main, develop] }
  pull_request: { branches: [main, develop] }
  schedule: [{ cron: "0 6 * * 1" }]
  workflow_dispatch:
permissions: { contents: read }

jobs:
  secret-scan:
    runs-on: ubuntu-latest
    if: ${ github.event_name != 'schedule' }
    continue-on-error: true
    steps:
      - uses: actions/checkout@v4
        with: { fetch-depth: 0 }
      - run: mkdir -p reports
      - name: Gitleaks
        run: docker run --rm -v "$PWD:/repo" zricethezav/gitleaks:v8.27.2 detect
          --source /repo --report-format json
          --report-path /repo/reports/gitleaks-report.json --exit-code 0 || true
      - name: TruffleHog
        run: docker run --rm -v "$PWD:/repo" ghcr.io/trufflesecurity/trufflehog:3.90.8
          filesystem /repo --results=verified,unknown --json
          > reports/trufflehog-report.json || true
      - uses: actions/upload-artifact@v4
        with: { name: secret-scan-reports, path: reports/ }

  sast:
    runs-on: ubuntu-latest
    if: ${ github.event_name != 'schedule' }
    continue-on-error: true
    steps:
      - uses: actions/checkout@v4
      - run: mkdir -p reports
      - name: Semgrep
        run: pipx run semgrep scan --config p/ci --config p/security-audit
          --config p/python --json --output reports/semgrep-report.json
          backend/src/finsight || true
      - name: Bandit
        run: pipx run bandit -r backend/src/finsight -f json
          -o reports/bandit-report.json || true
      - uses: actions/upload-artifact@v4
        with: { name: sast-reports, path: reports/ }

  sca:
    runs-on: ubuntu-latest
    continue-on-error: true
    steps:
      - uses: actions/checkout@v4
      - name: Trivy filesystem (deps)
        uses: aquasecurity/trivy-action@v0.36.0
        with: { scan-type: fs, scan-ref: ., format: json,
          output: reports/trivy-fs-report.json, severity: HIGH,CRITICAL }
      - name: Trivy SBOM (CycloneDX)
        uses: aquasecurity/trivy-action@v0.36.0
        with: { scan-type: fs, scan-ref: ., format: cyclonedx, output: reports/sbom.json }
      - uses: actions/upload-artifact@v4
        with: { name: sca-reports, path: reports/ }

  dast:
    runs-on: ubuntu-latest
    if: ${ github.event_name == 'schedule' || github.event_name == 'workflow_dispatch'
      || (github.event_name == 'push' && github.ref == 'refs/heads/main') }
    continue-on-error: true
    steps:
      - uses: actions/checkout@v4
      - run: docker compose up -d --build web db
      - name: Wait for /health
        run: for i in $(seq 1 30); do curl -sf http://localhost:8000/health && break; sleep 5; done
      - name: OWASP ZAP baseline
        uses: zaproxy/action-baseline@v0.12.0
        with: { target: 'http://localhost:8000', fail_action: false }

```

```

defectdojo-gate:
  runs-on: ubuntu-latest
  steps:
    - id: check
      env: { DEFECTDOJO_URL: '${{ secrets.DEFECTDOJO_URL }}' }
      run: echo "enabled=$( [ -n "$DEFECTDOJO_URL" ] && echo true || echo false )" >> "$GITHUB_OUTPUT"
      outputs: { enabled: '${{ steps.check.outputs.enabled }}' }

defectdojo-upload:
  runs-on: ubuntu-latest
  needs: [defectdojo-gate, secret-scan, sast, sca, dast]
  if: '${{ always() && needs.defectdojo-gate.outputs.enabled == 'true' }}'
  continue-on-error: true
  steps:
    - uses: actions/checkout@v4
    - uses: actions/download-artifact@v4
    - name: Upload reports to DefectDojo (reimport-scan)
      env:
        DEFECTDOJO_URL: '${{ secrets.DEFECTDOJO_URL }}'
        DEFECTDOJO_TOKEN: '${{ secrets.DEFECTDOJO_TOKEN }}'
        DEFECTDOJO_PRODUCT_ID: '${{ secrets.DEFECTDOJO_PRODUCT_ID }}'
      run: |
        bash scripts/upload_to_defectdojo.sh "Gitleaks Scan"          reports/gitleaks-report.json
        bash scripts/upload_to_defectdojo.sh "Semgrep JSON Report"    reports/semgrep-report.json
        bash scripts/upload_to_defectdojo.sh "Bandit Scan"            reports/bandit-report.json
        bash scripts/upload_to_defectdojo.sh "Trivy Scan FS"          reports/trivy-fs-report.json "Trivy
Scan"

```

Nota de implementación: `secrets` no está disponible en el `if`: a nivel de job en GitHub Actions; por eso el job `defectdojo-gate` expone la presencia del secreto como un *output* booleano que `defectdojo-upload` consume.

11. Lecciones aprendidas (justificación del uso de DevOps)

- 1. Problema de negocio:** FinSight maneja datos financieros personales (PII). Una filtración o un acceso indebido entre hogares destruye la propuesta *privacy-first* y la confianza del usuario.
- 2. Limitaciones antes:** la seguridad dependía de revisión manual y disciplina individual; las vulnerabilidades eran invisibles hasta producción y no había forma de medir la postura de seguridad.
- 3. Qué cambió con DevOps:** la seguridad se volvió *código* y parte del pipeline (*shift-left*). Cada push ejecuta SAST, DAST, SCA y secret scanning, y los hallazgos se centralizan automáticamente en DefectDojo.
- 4. Ventajas concretas:** detección temprana y automática de CVEs y secretos; trazabilidad y priorización de hallazgos; doble barrera contra secretos; y un proceso repetible que no depende de que alguien recuerde revisar. Además, 4 amenazas STRIDE fueron mitigadas en el código con TDD estricto (cobertura backend $\geq 90\%$).

12. Recomendaciones para implementar DevOps en otros proyectos

1. Empezar por *shift-left*: integrar secret scanning y SCA primero — baratos, rápidos y de alto impacto.
2. Doble barrera *pre-commit* + CI: detener problemas antes del push y revalidarlos en el pipeline.
3. Centralizar hallazgos (DefectDojo) para no perderlos en logs de CI y poder priorizar por severidad.

4. Modelar amenazas temprano (STRIDE): dirige qué controles importan según el contexto del negocio.
5. `allow_failure` al inicio y *gates* después: reportar sin romper el build, y endurecer a *fail-on-finding* cuando el equipo madura.
6. Tratar la seguridad como parte de la *Definition of Done*, no como una fase final aparte.

13. Entregables y enlaces

1. **Repositorio** (código + pipeline): github.com/bylenz/finsight
2. **Diagrama STRIDE** (draw.io): docs/informe-final/stride-finsight.drawio
3. **Plantilla STRIDE** (Excel): docs/informe-final/stride-finsight.xlsx
4. **Informe PDF**: este documento.
5. **Evidencia**: docs/informe-final/evidence/ (capturas DefectDojo).